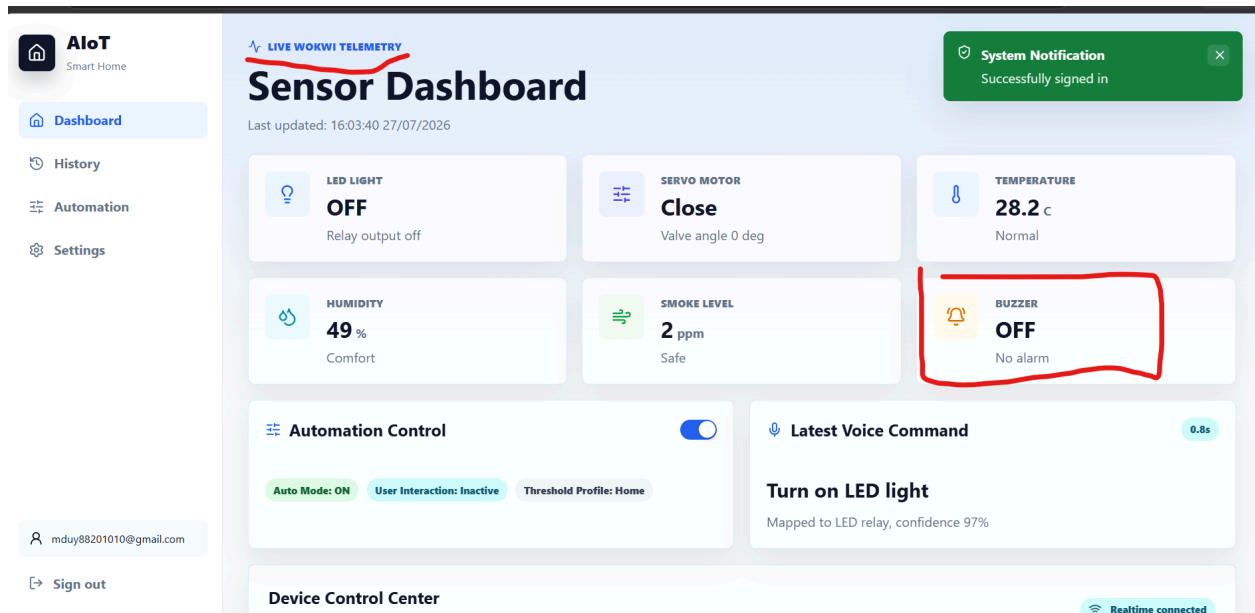
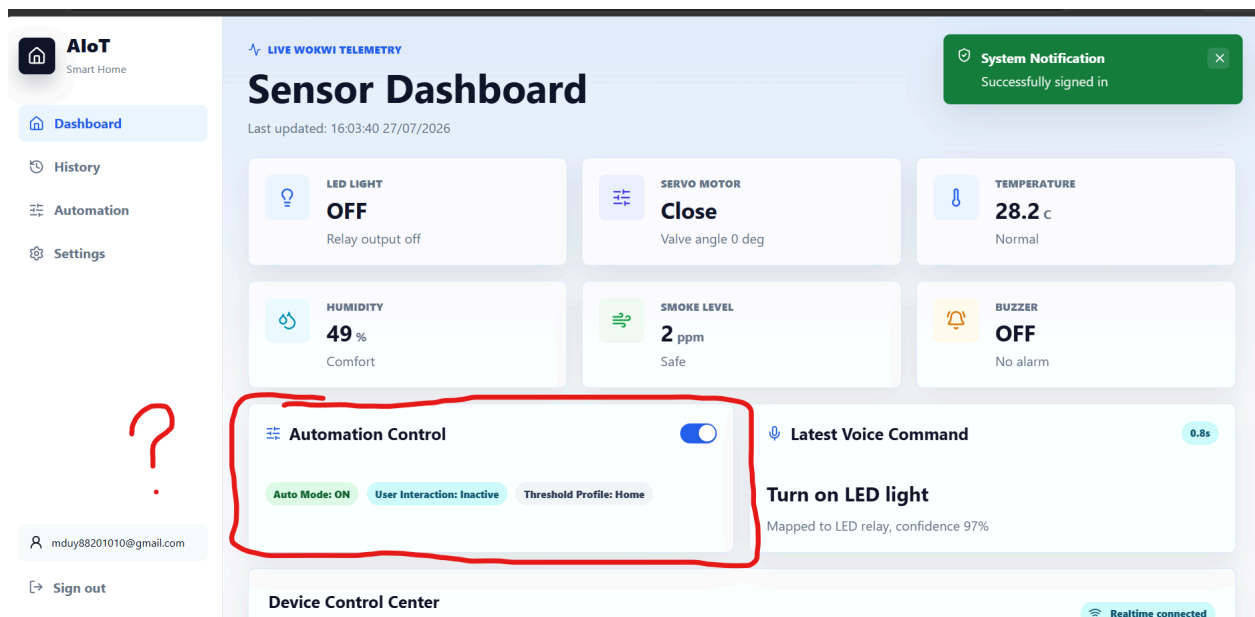
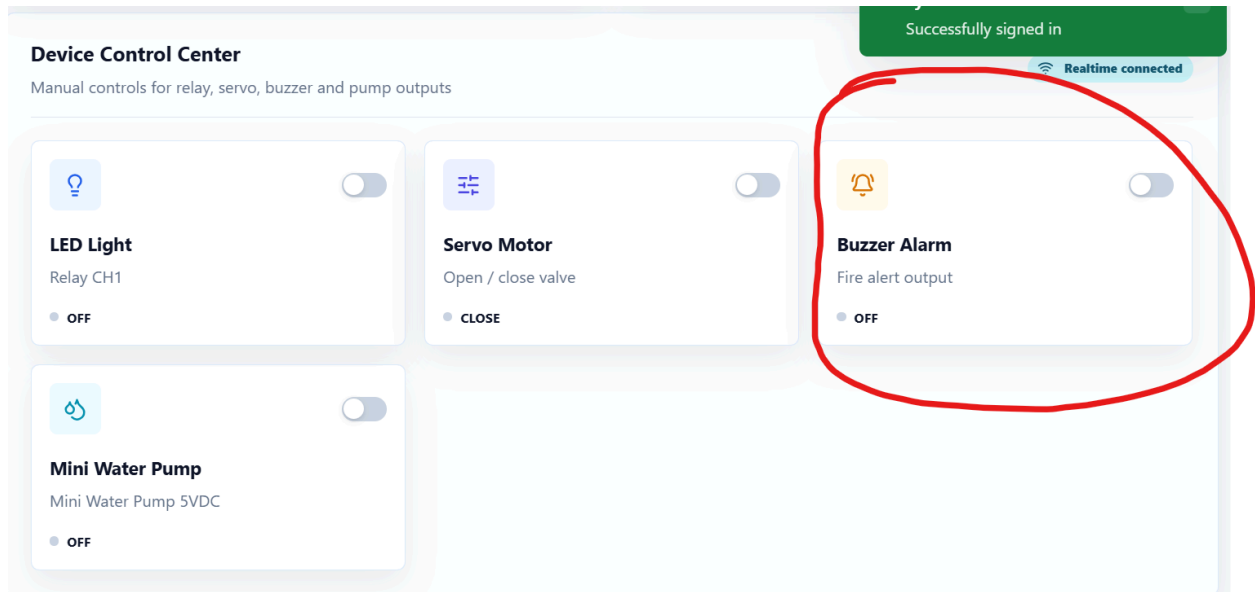




Đổi buzzer -> water pump (máy bơm)



Đoạn này có vẻ không cần thiết



Bỏ buzzer



Tại phần này nên chia ra thành 2 bảng duy nhất: bảng đầu tiên gồm nhiệt độ và độ ẩm. Cột Nhiệt độ nằm bên trái, cột độ ẩm nằm bên phải. Nhớ sử dụng Khoảng số phù hợp để hiển thị, ví dụ: nhiệt độ thường nằm trong khoảng 28 -> 30 độ thì để cột nhiệt độ từ 25 -> 35 độ.

Bảng thứ 2 theo dõi trạng thái nồng độ khói
LED, servo, buzzer không cần track trên bảng này

Phần 2: TRUY VẤN / TRẢ DỮ LIỆU LỊCH SỬ CHO FRONTEND (CHƯA CÓ ⚠)

Backend **chưa mở REST API để lấy chuỗi lịch sử (History range API)** cho Front-end:

- Hiện tại :TelemetryController.java mới chỉ có endpoint GET /api/telemetry/latest (trả về **1 bản ghi mới nhất**).
- Chưa có endpoint dạng GET /api/telemetry/history hay GET /api/logs/history để lấy danh sách/mảng dữ liệu theo thời gian (ví dụ 24h qua) phục vụ biểu đồ trên trang **History**.

Phần 3: Thêm các phần cần thiết để thay vì hiển thị mock data, sẽ lấy dữ liệu thật từ database. Phần copy sau lấy từ antigravity:

Implementation Plan - Replace Mock Data with Supabase Database History

Summary: Bổ sung các REST API truy vấn dữ liệu lịch sử trên Spring Boot Backend và cập nhật các component trên React Frontend để truy xuất và hiển thị dữ liệu lịch sử thực tế từ Supabase PostgreSQL (thay vì dùng dữ liệu giả từ `mockData.js`).

User Review Required

> [!IMPORTANT]

> - Backend sẽ bổ sung 2 API mới:

> 1. `GET /api/telemetry/history?limit=30`: Lấy danh sách nồng độ khói, nhiệt độ, độ ẩm ghi nhận từ ESP32 theo thời gian.

> 2. `GET /api/devices/history/logs?limit=30`: Lấy danh sách nhật ký điều khiển (bật/tắt đèn, mở/đóng cửa...).

> - Frontend sẽ cập nhật `SensorTimeline.jsx` (vẽ biểu đồ từ DB) và `HistoryPage.jsx` / `DashboardPage.jsx` (danh sách hoạt động gần đây từ DB).

Open Questions

Không có.

Proposed Changes

Backend (Spring Boot)

[MODIFY]

[TelemetryRepository.java](file:///e:/DISK_D/MDUY/Nam3/3.2%20AIoT/Proj/AIOT-Smart-Home/backend/src/main/java/com/aiot/smarthome/repository/TelemetryRepository.java)

- Thêm hàm `findHistory(int limit)` để truy vấn `SELECT temperature, humidity, smoke\_ppm, measured\_at FROM telemetry\_readings ORDER BY measured\_at DESC LIMIT ?`.

[MODIFY]

[TelemetryService.java](file:///e:/DISK_D/MDUY/Nam3/3.2%20AIoT/Proj/AIOT-Smart-Home/backend/src/main/java/com/aiot/smarthome/service/TelemetryService.java)

- Bổ sung hàm `getTelemetryHistory(int limit)` trả về danh sách `TelemetryResponse`.

[MODIFY]

[TelemetryController.java](file:///e:/DISK_D/MDUY/Nam3/3.2%20AIoT/Proj/AIOT-Smart-Home/backend/src/main/java/com/aiot/smarthome/controller/TelemetryController.java)

- Bổ sung endpoint `@GetMapping("/history")` với query parameter `@RequestParam(defaultValue = "30") int limit`.

[MODIFY]

[DeviceRepository.java](file:///e:/DISK_D/MDUY/Nam3/3.2%20AIoT/Proj/AIOT-Smart-Home/backend/src/main/java/com/aiot/smarthome/repository/DeviceRepository.java)

- Thêm hàm `findControlLogs(int limit)` để truy vấn `SELECT device\_id, requested\_state, source, result, created\_at FROM control\_logs ORDER BY created\_at DESC LIMIT ?`.

[MODIFY]

[DeviceService.java](file:///e:/DISK_D/MDUY/Nam3/3.2%20AIoT/Proj/AIoT-Smart-Home/backend/src/main/java/com/aiot/smarthome/service/DeviceService.java) &

[DeviceController.java](file:///e:/DISK_D/MDUY/Nam3/3.2%20AIoT/Proj/AIoT-Smart-Home/backend/src/main/java/com/aiot/smarthome/controller/DeviceController.java)

- Bổ sung endpoint `@GetMapping("/history/logs")` trả về danh sách nhật ký điều khiển thiết bị.

Frontend (React + Vite)

[MODIFY]

[api.js](file:///e:/DISK_D/MDUY/Nam3/3.2%20AIoT/Proj/AIoT-Smart-Home/frontend/src/lib/api.js)

- Thêm hàm `getTelemetryHistory(limit)` gọi `GET /api/telemetry/history`.

- Thêm hàm `getControlLogs(limit)` gọi `GET /api/devices/history/logs`.

[MODIFY]

[SensorTimeline.jsx](file:///e:/DISK_D/MDUY/Nam3/3.2%20AIoT/Proj/AIoT-Smart-Home/frontend/src/components/SensorTimeline.jsx)

- Đổi từ việc import tĩnh `chartSeries` từ `mockData.js` sang việc gọi `getTelemetryHistory()` khi component mount và bổ sung điểm dữ liệu mới mỗi khi nhận `TELEMETRY` qua WebSocket.

[MODIFY]

[DashboardPage.jsx](file:///e:/DISK_D/MDUY/Nam3/3.2%20AIoT/Proj/AIoT-Smart-Home/

e/frontend/src/components/DashboardPage.jsx) &
[HistoryPage.jsx](file:///e:/DISK_D/MDUY/Nam3/3.2%20AIoT/Proj/AIoT-Smart-Home/frontend/src/components/HistoryPage.jsx)

- Đổi từ `recentActivityRows` của `mockData.js` sang việc gọi `getControlLogs()` để hiển thị nhật ký thao tác thực tế từ Database.

Verification Plan

Automated Tests

- Chạy `.mvnw.cmd test` hoặc kiểm tra build Spring Boot package `.mvnw.cmd package -DskipTests`.

Manual Verification

1. Đăng nhập ứng dụng React Frontend.
2. Kiểm tra xem biểu đồ trong `SensorTimeline` đã hiển thị các điểm dữ liệu lịch sử đọc từ bảng `telemetry\_readings` của Supabase Postgres chưa.
3. Thử bật/tắt thiết bị (LED/Servo), kiểm tra xem bảng nhật ký Recent Activity trong Dashboard và trang History có xuất hiện bản ghi tương ứng từ DB hay chưa.